

JavaScript Interview Concepts (Most Asked)

This version gives a little more detail for each concept, with a quick example you can use in interviews.

1) `var`, `let`, `const`

What it means: `var` is function-scoped and can be redeclared; `let` and `const` are block-scoped. `const` prevents reassignment (not deep mutation).

Example:

```
var a = 1;
let b = 2;
const c = { score: 10 };
c.score = 20; // allowed
// c = {} // not allowed
```

2) Scope and Lexical Scope

What it means: Scope controls variable visibility. Lexical scope means inner functions access variables from where they were defined.

Example:

```
function outer() {
  const msg = "hello";
  function inner() {
    console.log(msg);
  }
  inner();
}
```

3) Hoisting

What it means: Declarations are processed before execution. `var` is initialized to `undefined`; `let`/`const` are hoisted but uninitialized.

Example:

```
console.log(x); // undefined
var x = 10;
```

4) Temporal Dead Zone (TDZ)

What it means: From block start until `let/const` declaration line, accessing the variable throws `ReferenceError`.

Example:

```
// console.log(y); // ReferenceError
let y = 20;
```

5) Primitive vs Reference Types

What it means: Primitives copy by value; objects/arrays/functions copy by reference.

Example:

```
let p1 = 5;
let p2 = p1;
p2 = 9; // p1 still 5

const o1 = { n: 1 };
const o2 = o1;
o2.n = 7; // o1.n is also 7
```

6) Type Coercion (== vs ===)

What it means: `==` coerces types; `===` compares type and value exactly.

Example:

```
0 == false; // true
0 === false; // false
```

7) Truthy and Falsy

What it means: JavaScript converts values in boolean contexts. Falsy values are only: `false`, `0`, `' '`, `null`, `undefined`, `NaN`.

Example:

```
if ("0") console.log("truthy");
if (0) console.log("never runs");
```

8) Closures

What it means: A function remembers variables from its outer scope even after outer function returns.

Example:

```
function counter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
const inc = counter();  
console.log(inc()); // 1  
console.log(inc()); // 2
```

9) `this` Keyword

What it means: `this` depends on how a function is called, not where it is written.

Example:

```
const user = {  
  name: "Ava",  
  greet() {  
    console.log(this.name);  
  },  
};  
user.greet(); // Ava
```

10) Arrow Function vs Regular Function

What it means: Arrow functions inherit `this` lexically; regular functions get their own `this` based on call-site.

Example:

```
const obj = {  
  id: 1,  
  regular: function () {  
    return this.id;  
  },  
  arrow: () => this.id,  
};  
console.log(obj.regular()); // 1  
console.log(obj.arrow()); // usually undefined
```

11) Execution Context and Call Stack

What it means: JS runs code in execution contexts and tracks active calls in a stack (LIFO).

Example:

```
function a() {  
  b();  
}  
function b() {  
  c();  
}  
function c() {  
  console.log("top of stack");  
}  
a();
```

12) Event Loop

What it means: JS is single-threaded; async callbacks are queued. Microtasks run before next macrotask.

Example:

```
console.log("A");  
setTimeout(() => console.log("B"), 0);  
Promise.resolve().then(() => console.log("C"));  
console.log("D");  
// Output: A D C B
```

13) Promises

What it means: Promise represents future completion/failure of async work with states: pending, fulfilled, rejected.

Example:

```
fetch("/api")  
  .then((res) => res.json())  
  .then((data) => console.log(data))  
  .catch((err) => console.error(err));
```

14) `async` / `await`

What it means: Cleaner syntax over promise chains; still asynchronous and returns a promise.

Example:

```
async function getData() {  
  try {
```

```
const res = await fetch("/api");
const data = await res.json();
return data;
} catch (e) {
  console.error(e);
}
}
```

15) `setTimeout` and `setInterval`

What it means: Timers schedule callbacks, but exact timing can drift due to event loop and main-thread load.

Example:

```
setTimeout(() => console.log("once"), 500);
const id = setInterval(() => console.log("repeat"), 1000);
setTimeout(() => clearInterval(id), 3500);
```

16) Debounce and Throttle

What it means: Debounce waits for pause; throttle limits rate to once per interval.

Example:

```
const debounced = debounce(searchApi, 400);
const throttled = throttle(onScroll, 200);
```

17) Array Methods You Must Know

What it means: `map` transforms, `filter` selects, `reduce` accumulates.

Example:

```
const nums = [1, 2, 3, 4];
const evenSquares = nums.filter((n) => n % 2 === 0).map((n) => n * n);
const total = nums.reduce((sum, n) => sum + n, 0);
```

18) `map` vs `forEach`

What it means: `map` returns a new array; `forEach` is for side effects and returns `undefined`.

Example:

```
const arr = [1, 2, 3];
const doubled = arr.map((n) => n * 2);
arr.forEach((n) => console.log(n));
```

19) Shallow Copy vs Deep Copy

What it means: Spread/Object.assign make shallow copies; nested references stay shared.

Example:

```
const original = { nested: { x: 1 } };
const shallow = { ...original };
shallow.nested.x = 9;
console.log(original.nested.x); // 9
```

20) Object and Array Destructuring

What it means: Pull values from objects/arrays into variables concisely.

Example:

```
const user = { name: "Ava", age: 22 };
const { name } = user;
const [first, second] = [10, 20];
```

21) Rest and Spread

What it means: Rest collects many values; spread expands iterable/object values.

Example:

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
const merged = [...[1, 2], ...[3, 4]];
```

22) Template Literals

What it means: Use backticks for interpolation and multiline strings.

Example:

```
const name = "Ava";
const msg = `Hello, ${name}`;
```

23) Optional Chaining and Nullish Coalescing

What it means: `?.` safely accesses nested properties. `??` defaults only for `null/undefined`.

Example:

```
const city = user?.address?.city;
const page = inputPage ?? 1;
```

24) Prototypal Inheritance

What it means: Objects inherit through prototype chain; lookup climbs chain when property missing.

Example:

```
const animal = {
  eat() {
    return "eating";
  },
};
const dog = Object.create(animal);
console.log(dog.eat()); // eating
```

25) Classes in JavaScript

What it means: `class` syntax wraps prototype-based behavior in cleaner form.

Example:

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hi, I am ${this.name}`;
  }
}
```

26) `call`, `apply`, `bind`

What it means: Control `this` explicitly. `call/apply` invoke now; `bind` returns new function.

Example:

```
function greet(city) {  
  console.log(this.name, city);  
}  
const user = { name: "Ava" };  
greet.call(user, "Delhi");  
greet.apply(user, ["Delhi"]);  
const bound = greet.bind(user);  
bound("Delhi");
```

27) Higher-Order Functions

What it means: Functions that take or return functions.

Example:

```
function withLog(fn) {  
  return (...args) => {  
    console.log("calling");  
    return fn(...args);  
  };  
}
```

28) Currying

What it means: Convert multi-arg function into sequence of single-arg functions.

Example:

```
const multiply = (a) => (b) => a * b;  
const double = multiply(2);  
console.log(double(5)); // 10
```

29) Pure Functions and Side Effects

What it means: Pure function depends only on inputs and does not alter external state.

Example:

```
const pureAdd = (a, b) => a + b;  
let total = 0;  
function impureAdd(x) {  
  total += x;  
}
```

30) Immutability

What it means: Return new structures instead of mutating old ones; easier debugging and state management.

Example:

```
const state = { count: 1 };
const next = { ...state, count: state.count + 1 };
```

31) Modules (**import** / **export**)

What it means: Split code into reusable files with explicit public API.

Example:

```
// math.js
export const add = (a, b) => a + b;

// app.js
import { add } from "./math.js";
```

32) CommonJS vs ES Modules

What it means: CommonJS is **require/module.exports**; ESM is **import/export** and is the modern standard.

Example:

```
// CommonJS
const fs = require("fs");

// ESM
import fs from "fs";
```

33) Error Handling

What it means: Handle expected failures gracefully and keep app stable.

Example:

```
try {
  JSON.parse("{bad json}");
} catch (e) {
  console.error("Invalid JSON");
}
```

34) Memory Leaks (Common Causes)

What it means: Objects remain referenced unintentionally, so garbage collector cannot free them.

Example:

```
const handler = () => console.log("resize");
window.addEventListener("resize", handler);
// later, cleanup is required:
window.removeEventListener("resize", handler);
```

35) DOM Event Propagation

What it means: Events flow capture -> target -> bubble.

Example:

```
document.getElementById("child").addEventListener("click", (e) => {
  e.stopPropagation();
});
```

36) Event Delegation

What it means: One parent listener handles many child elements using `event.target`.

Example:

```
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.matches("button.remove")) {
    console.log("remove clicked");
  }
});
```

37) `localStorage`, `sessionStorage`, Cookies

What it means: Browser storage options differ in lifetime and request behavior.

Example:

```
localStorage.setItem("theme", "dark");
sessionStorage.setItem("step", "2");
document.cookie = "token=abc; Max-Age=3600; Secure";
```

38) `== null` Trick

What it means: `value == null` is true for both `null` and `undefined` only.

Example:

```
let value;
if (value == null) {
  console.log("null or undefined");
}
```

39) Strict Mode

What it means: Enables safer JS behavior and catches silent mistakes.

Example:

```
"use strict";
// x = 10; // ReferenceError instead of creating global
```

40) Big O Basics for JS Interviews

What it means: Time/space complexity communicates scalability of your solution.

Example:

```
// O(n)
function findUser(users, id) {
  return users.find((u) => u.id === id);
}
```

Quick Rapid-Fire Questions (Practice)

1. Difference between `null` and `undefined`?
2. Explain closure with one real use case.
3. Why does arrow function `this` differ from regular function `this`?
4. Microtask vs macrotask: execution order?
5. Debounce vs throttle with one practical example each.
6. `map` vs `forEach` in one sentence.
7. `call` vs `apply` vs `bind` difference.
8. Why can shallow copy cause bugs?
9. Promise chain vs `async/await` trade-off.
10. Name three common causes of memory leaks.

Interview Answer Formula

1. Define in one line.

2. Give one practical example.
3. Mention one pitfall.
4. Mention one best practice.
5. If coding is requested, start simple then optimize.